

This section demonstrates real-world team collaboration through a structured four-day development process where agents work together to build a simple todo list application. Participants manage multiple terminal sessions, each running a different agent (Product Manager, QA Tester, Python Developer), while the workshop facilitator acts as the UAT User providing stakeholder feedback and approval gates. The demo progresses through requirements definition, development implementation, comprehensive testing with pytest, and final acceptance, showcasing how agents collaborate through shared files, respect role boundaries, and adapt their behavior based on stakeholder input to deliver a working application.

## Terminal Setup & Management

### Separate Terminal Windows

#### 1. Terminal 1 – Product Manager

```
1 cd simple-todo-app && \
2 kiro-c11 chat --agent product-manager
```

#### 2. Terminal 2 – QA Tester

```
1 cd simple-todo-app && \
2 kiro-c11 chat --agent qa-tester
```

#### 3. Terminal 3 – Python Developer

```
1 cd simple-todo-app && \
2 kiro-c11 chat --agent python-developer
```

#### 4. Terminal 4 – UAT User (You – Workshop Facilitator)

You'll manually review files and provide feedback

```
1 cd simple-todo-app
```

### Checkpoint: Verify All Agents Load

Before starting the development, test each agent with this prompt:

```
1 Tell me your role
```

#### 1. Terminal 1 – Product Manager

Expected: I'm Product Manager Bob...

#### 2. Terminal 2 – QA Tester

Expected: I'm QA Tester Quinn...

#### 3. Terminal 3 – Python Developer

Expected: I'm Python Developer Dave...

## Develop Simple Todo List App

### Round 1: Requirements Definition

**Round 1** focuses on establishing clear project requirements through stakeholder collaboration. The **Product Manager** writes business requirements in `shared/requirements.md`, defining what the todo app should do without technical details, while the **UAT User** reviews and provides feedback until writing "APPROVED" status. The **Python Developer** and **QA Tester** remain inactive, waiting for approval. This round serves as a critical gate control—development cannot begin until the UAT User explicitly approves the requirements, ensuring all subsequent work builds on an agreed-upon foundation.

Product Manager (Bob)

**PRIMARY ROLE:** Requirements Creator

- Write clear, testable requirements in `shared/requirements.md`
- Focus on WHAT the app should do (user functionality)
- Avoid HOW (no technical implementation details)
- Respond to UAT feedback and update requirements accordingly
- Make business decisions about scope and priorities

**Key Output:** Business requirements document

UAT User (YOU – Workshop Facilitator)

**PRIMARY ROLE:** Requirements Reviewer & Gatekeeper

- Review requirements in `shared/requirements.md` for clarity
- Provide feedback in `shared/uat-feedback.md`
- Ask questions about unclear or missing requirements
- Make final approval decision: Write "APPROVED" or "NEEDS REVISION"
- Gate control: Development cannot start without "APPROVED" status

**Key Output:** Stakeholder approval decision

File Interactions:

- `shared/requirements.md`: PM writes, UAT reviews
- `shared/uat-feedback.md`: UAT writes, PM responds

#### 1. Product Manager (Terminal 1):

Prompt:

```
1 I need to define requirements for a simple todo list application. Create clear, testable requirements in shared/requirements.md that include:
2
3 1. Users can add a new task with text
4 2. Users can mark tasks as complete/incomplete
5 3. Users can delete tasks
6 4. Users can view all their tasks
7
8 Keep it simple - no fancy features, just core functionality that works. Focus on WHAT the app should do, not HOW to build it. Let the developer handle all the technical details.
```

**Expected Output:** Product Manager creates simple, clear requirements focused on user functionality, NOT technical implementation.

Important:

If Product Manager tries to suggest code or technical solutions, this demonstrates the need for clear role boundaries.

#### 2. UAT User (You – Workshop Facilitator):

After Product Manager finishes, review `shared/requirements.md` and provide feedback:

Review the requirements

```
1 cat shared/requirements.md
```

Create feedback file

```
1 cat > shared/uat-feedback.md << EOF
2 ## UAT Feedback on Requirements
3
4 ### What I Like:
5 - [Your positive feedback]
6
7 ### Questions/Concerns:
8 - [Any unclear requirements]
9 - [Missing details]
10
11 ### Acceptance:
12 - [ ] Requirements are clear and testable
13 - [ ] Requirements cover basic todo functionality
14 - [ ] Ready for development
15
16 Status: [APPROVED/NEEDS REVISION]
17
18 Date: $(date)
19
20 EOF
```

#### 3. Product Manager (Terminal 1) – Iteration:

After UAT provides feedback, review and respond:

Prompt:

```
1 Check UAT feedback in shared/uat-feedback.md and respond accordingly:
2
3 If Status is "NEEDS REVISION":
4 - Read the UAT questions and concerns carefully
5 - Update shared/requirements.md to address their feedback
6 - Make requirements clearer and more specific
7 - Add any missing details they identified
8 - Notify UAT when ready for re-review
9
10 If Status is "APPROVED":
11 - Requirements are finalized and approved by stakeholder
12 - Development can begin in Round 2
13 - No further changes needed to requirements
```

#### 4. UAT User (You – Workshop Facilitator) – Final Approval:

If PM made changes, review again and provide final approval:

Review updated requirements (if PM made changes)

```
1 cat shared/requirements.md
```

Update your feedback with final decision

```
1 cat > shared/uat-feedback.md << EOF
2 ## FINAL UAT APPROVAL
3
4 Status: APPROVED
5 Ready for development: YES
6
7 Date: $(date)
8
9 EOF
```

### Round 2: Development Phase

**Round 2** begins once requirements are UAT-approved, with the **Python Developer** taking the lead to build the todo list web application. Working from the approved requirements in `shared/requirements.md`, the developer creates a Flask/FastAPI application in the isolated `src/todo-app/` environment, implementing core functionality like adding, completing, and deleting tasks. Throughout development, the developer updates `shared/status.md` with progress reports and signals when features are ready for testing. The **Product Manager**, **QA Tester**, and **UAT User** monitor progress but remain inactive during this phase, allowing the developer to focus on technical implementation without interference.

Python Developer (Dave)

**PRIMARY ROLE:** Application Builder

- Read approved requirements from `shared/requirements.md`
- Build Flask/FastAPI web application in `src/todo-app/`
- Implement add, complete, delete task functionality
- Update progress in `shared/status.md`
- Signal when features are ready for testing

**Key Output:** Working web application

File Interactions:

- `shared/requirements.md`: Developer reads (approved requirements)
- `shared/status.md`: Developer writes (progress updates)

#### 1. Python Developer (Terminal 3):

Prompt:

```
1 Review the requirements from shared/requirements.md (approved by UAT in Round 1).
2
3 Build a simple todo list web application:
4 1. Activate the application environment: cd src/todo-app && source venv/bin/activate
5 2. Create a web app using Flask or FastAPI
6 3. Create HTML templates for the user interface
7 4. Implement add/complete/delete task functionality
8 5. Keep it simple - focus on working functionality
9 6. Update shared/status.md with your progress
10
11 Create the files in the src/todo-app/ directory and let the team know when features are ready for testing.
```

**Expected Output:** Developer creates basic working web application in isolated environment and updates status.

Tip:

If Developer gets stuck, remind them they have full access to `src/todo-app/` directory and their own virtual environment with Flask installed.

### Round 3: Testing Phase

**Round 3** activates once development is complete, with the **QA Tester** leading comprehensive application testing. Using the approved requirements as test criteria and checking `shared/status.md` for ready features, the QA Tester creates pytest-based test suites with fixtures, parametrized tests, and coverage reporting in the isolated `tests/` environment. All testing results are documented in `shared/test-report.md`, while any discovered bugs are logged in `shared/bugs.md` with detailed reproduction steps. The **Python Developer** monitors bug reports for upcoming fixes, while the **Product Manager** and **UAT User** await testing completion before final evaluation.

QA Tester (Quinn)

**PRIMARY ROLE:** Quality Assurance Lead

- Create pytest test suites with fixtures and parametrized tests
- Test application against requirements in isolated `tests/` environment
- Run comprehensive testing with coverage reporting
- Document results in `shared/test-report.md`
- Log bugs in `shared/bugs.md` with reproduction steps

**Key Output:** Test results and bug reports

File Interactions:

- `shared/requirements.md`: QA Tester reads (test criteria)
- `shared/status.md`: QA Tester reads (ready features)
- `shared/test-report.md`: QA Tester writes (pytest results, coverage)
- `shared/bugs.md`: QA Tester writes (bug reports)

1. QA Tester (Terminal 2):

Prompt:

```
1 Review the requirements from shared/requirements.md and check what the developer has completed in shared/status.md.
2
3 Implement comprehensive pytest-based testing for the todo web application:
4
5 1. "Activate testing environment:" cd tests && source venv/bin/activate
6
7 2. "Create pytest test structure:"
8 - Create tests/conftest.py with fixtures for API client and server setup
9 - Create tests/test_todo_api.py with parametrized tests for all API endpoints
10 - Create tests/test_todo_validation.py for edge cases and validation testing
11
12 3. "Implement pytest best practices:"
13 - Use pytest.fixture for test setup and teardown
14 - Use pytest.mark.parametrize for testing multiple scenarios
15 - Use requests library within pytest functions for API testing
16 - Add proper assertions and error handling
17
18 4. "Create manual test procedures:"
19 - Write tests/manual_test_steps.md with step-by-step UI testing procedures
20
21 5. "Run comprehensive test suite:"
22 - Execute: pytest -v --cov=./src/todo-app
23 - Generate coverage report: pytest --cov-report=html
24 - Document all results in shared/test-report.md
25 - Log any bugs found in shared/bugs.md
26
27 Focus on pytest as the primary testing framework, only write custom Python scripts when pytest cannot handle specific testing needs.
```

**Expected Output:** QA Tester creates comprehensive pytest-based test suite with fixtures, parametrized tests, and coverage reporting in isolated testing environment.

### Round 4: Bug Fixes & Iteration

**Round 4** represents the final project gate, where the **UAT User** conducts acceptance testing and makes the ultimate go/no-go decision. After reviewing `shared/status.md` and `shared/test-report.md`, the UAT User tests the application from an end-user perspective and writes either "ACCEPTED" or "REJECTED" in `shared/uat-feedback.md`. If rejected, the **Product Manager** coordinates team response and prioritization, the **Python Developer** fixes critical issues, and the **QA Tester** verifies fixes through regression testing. This creates an iterative cycle that continues until the UAT User writes "ACCEPTED", signaling project completion and readiness for users.

UAT User (YOU – Workshop Facilitator)

**PRIMARY ROLE:** Final Decision Maker

- Review `shared/status.md` and `shared/test-report.md`
- Test application from end-user perspective
- Make final decision: Write "ACCEPTED" or "REJECTED"
- Provide specific feedback on what needs fixing (if rejected)

**Key Output:** Final acceptance decision

Product Manager (Bob)

**PRIMARY ROLE:** Team Coordinator (if rejected)

- Coordinate team response to UAT feedback
- Prioritize critical fixes for acceptance
- Make business decisions about scope vs timeline

**Key Output:** Team coordination and priorities

Python Developer (Dave)

**PRIMARY ROLE:** Bug Fixer (if rejected)

- Fix critical issues identified by UAT
- Update `shared/status.md` with fix progress
- Signal QA when fixes are ready for re-testing

**Key Output:** Bug fixes and status updates

QA Tester (Quinn)

**PRIMARY ROLE:** Fix Verifier (if rejected)

- Re-test bug fixes reported by developer
- Run regression testing to ensure no new bugs
- Update `shared/test-report.md` with verification results
- Confirm fixes meet UAT acceptance criteria

**Key Output:** Fix verification and regression test results

File Interactions:

- `shared/requirements.md`: UAT User reads (acceptance criteria)
- `shared/status.md`: UAT User reads, Developer writes (if rejected)
- `shared/test-report.md`: UAT User reads, QA Tester writes (if rejected)
- `shared/bugs.md`: Developer reads (if rejected)
- `shared/uat-feedback.md`: UAT User writes (final decision)

#### 1. Python Developer (Terminal 3):

Prompt:

```
1 Review the bugs reported in shared/bugs.md.
2
3 Fix any critical issues and update shared/status.md with:
4 - Which bugs you've fixed in the code
5 - Any remaining issues
6 - Status of the application
7
8 Let the team know when fixes are ready for re-testing.
```

#### 2. QA Tester (Terminal 2):

Prompt:

```
1 Re-test any bug fixes reported by the developer in shared/status.md.
2
3 Run your pytest test suite to verify fixes:
4 1. Activate testing environment: cd tests && source venv/bin/activate
5 2. Execute: pytest -v --tb=short
6 3. Run specific tests for the bugs that were fixed: pytest -k "test_bug_fix"
7 4. Run regression testing: pytest --cov=./src/todo-app --cov-report=term-missing
8 5. Test manually if pytest tests don't cover the bug area
9 6. Verify no new bugs were introduced (regression testing)
10
11 Update shared/test-report.md with:
12 - Pytest test results with pass/fail counts
13 - Coverage report showing code coverage percentage
14 - Verification of bug fixes using specific pytest test cases
15 - Results of regression testing
16 - Any new issues found
17 - Overall application status and test coverage metrics
```

#### 3. UAT User (You – Workshop Facilitator):

Final acceptance testing:

Review all updates

```
1 cat shared/status.md
2 cat shared/test-report.md
```

Manual test follow tests/manual\_test\_steps.md

Final UAT decision

```
1 cat > shared/uat-feedback.md << EOF
2 ## FINAL UAT RE-EVALUATION
3
4 ### Application Review:
5 - Requirements met: [YES/NO with details]
6 - Quality acceptable: [YES/NO with details]
7 - Ready for release: [YES/NO]
8
9 ### Final Status: [ACCEPTED/REJECTED]
10
11 ### If Rejected:
12 - [List what needs to be fixed]
13
14 ### If Accepted:
15 - [Congratulate the team]
16 - Application is ready for users!
17
18 Date: $(date)
19 UAT Tester: Workshop Facilitator
20
21 EOF
```

4. Product Manager (Terminal 1) – Final Decision Response:

Prompt:

```
1 Check UAT final decision in shared/uat-feedback.md and coordinate accordingly:
2
3 If Status is "REJECTED":
4 - Review the specific issues UAT identified
5 - Prioritize which fixes are most critical for acceptance
6 - Coordinate with Developer and QA for another iteration
7 - Make business decisions about scope vs timeline
8 - Update team on priorities and next steps
9
10 If Status is "ACCEPTED":
11 - Congratulate the team on successful delivery
12 - Project is complete and ready for users
13 - Document lessons learned for future projects
```

**If REJECTED – Additional Iteration Cycle:**

Python Developer (Terminal 3) – Critical Fixes:

```
1 If UAT rejected the application, focus on critical fixes:
2 - Read UAT feedback in shared/uat-feedback.md for specific issues
3 - Read PM priorities and guidance for what to fix first
4 - Fix the most critical issues identified by UAT
5 - Update shared/status.md with fix progress
6 - Focus on acceptance criteria, not additional features
```

QA Tester (Terminal 2) – Re-verification:

```
1 If Developer made critical fixes, verify them:
2 - Test the specific issues UAT identified
3 - Run focused pytest tests on fixed areas
4 - Verify fixes don't break existing functionality
5 - Update shared/test-report.md with verification results
6 - Confirm fixes meet UAT acceptance criteria
```

UAT User (You – Workshop Facilitator) – Final Re-test:

If fixes were made, test again

```
1 cat shared/status.md
2 cat shared/test-report.md
```

Final decision after fixes

```
1 cat > shared/uat-feedback.md << EOF
2 ## FINAL UAT RE-EVALUATION
3
4 ### Fix Verification:
5 - Critical issues resolved: [YES/NO]
6 - Application now acceptable: [YES/NO]
7
8 ### Final Status: [ACCEPTED/REJECTED]
9
10 Date: $(date)
11 UAT Tester: Workshop Facilitator
12
13 EOF
```

### Checkpoint: Verify Collaboration

After each round, quickly check:

1. In a separate terminal, monitor file changes

```
1 watch -n 2 'ls -la shared/ && echo "---" && tail -5 shared/status.md'
```

Or manually check collaboration

```
1 # Expected: Files should have recent timestamps showing agent activity
2 ls -la shared/
```

2. Check UAT feedback integration

```
1 # Expected: Should show your feedback and team responses
2 cat shared/uat-feedback.md
```